

COMS 6998 AI-Native Computing: Hardware for AI and AI for Hardware

Week 1: Introduction

Zishen Wan

Assistant Professor, Department of CS

Columbia University

<https://wan-research-group.github.io/coms6998-f26/>

About me



Background

- Asst. Prof., Columbia CS (2026 fall - now)
- Postdoc, Harvard EECS (2026)
- Ph.D., Georgia Tech ECE (2025)

Office: CSB 522

Office Hours: Wed 5-6 PM

Research Interest

- Computer Architecture
- ML Systems
- VLSI Design
- Software-Hardware Co-Design
- System-on-Chip
- Physical AI & Agentic AI
- AI for Computing System Design

Faculty Profile:

<https://zishenwan.github.io/>

Research Group:

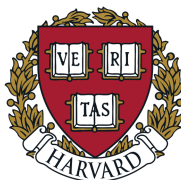
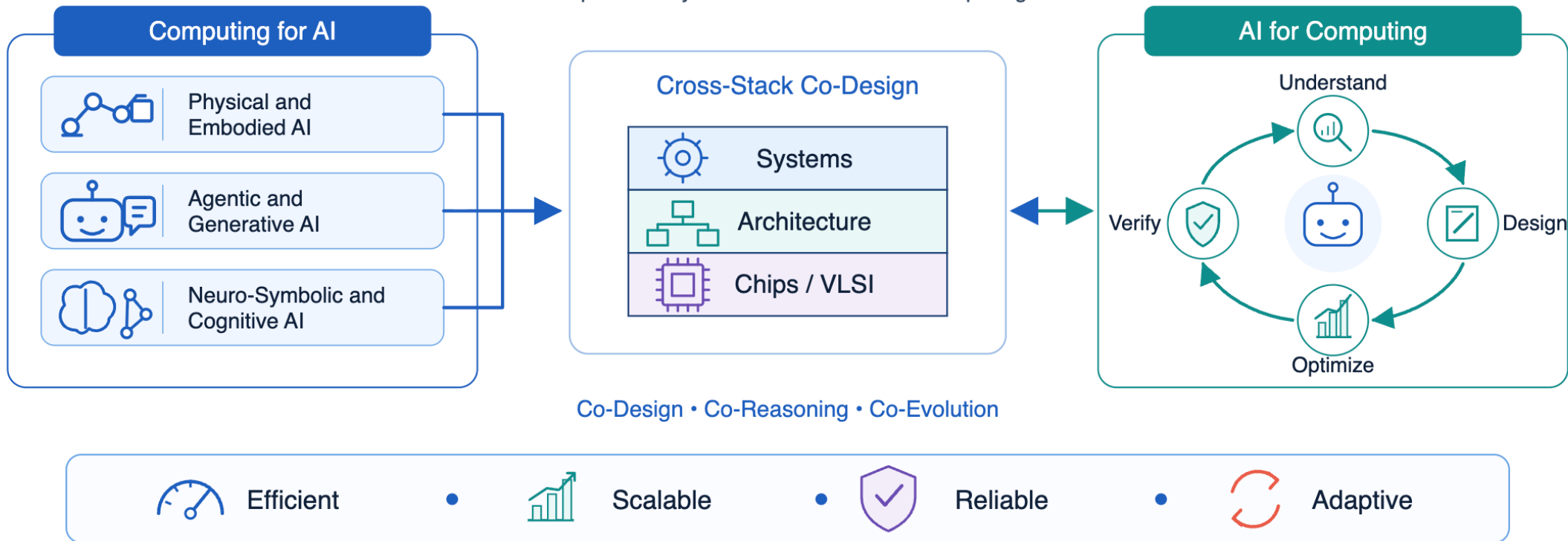
<https://wan-research-group.github.io/>

Announcement: we get 1-2 TA spot available for this course, reach out to me if you are interested

Wan Research Group

Computing for AI and AI for Computing

Two Complementary Directions Across the Computing Stack





I will try to remember everyone's name

**Please say your name first before asking or
answering questions**

Next, Your Turn



Introduce yourself to others at your table

5 min

- name and where you are from
- program, year and background
- research interest
- why you take this class and what you hope to learn

Each group selects a leader to introduce everyone at the table

Agenda for Today

- Course Overview
- Course Logistics
- Introduction to AI-Native Computing: HW for AI and AI for HW

Agenda for Today

- **Course Overview**
- Course Logistics
- Introduction to AI-Native Computing: HW for AI and AI for HW

1

Part 1

Why AI-Native Computing?

AI is the workload. AI is becoming the designer.

AI as a Workload & AI as a Designer



Introducing Redwood: A frontier AI accelerator designed, verified, and deployed from scratch in two weeks by AI

Research · August 27, 2026

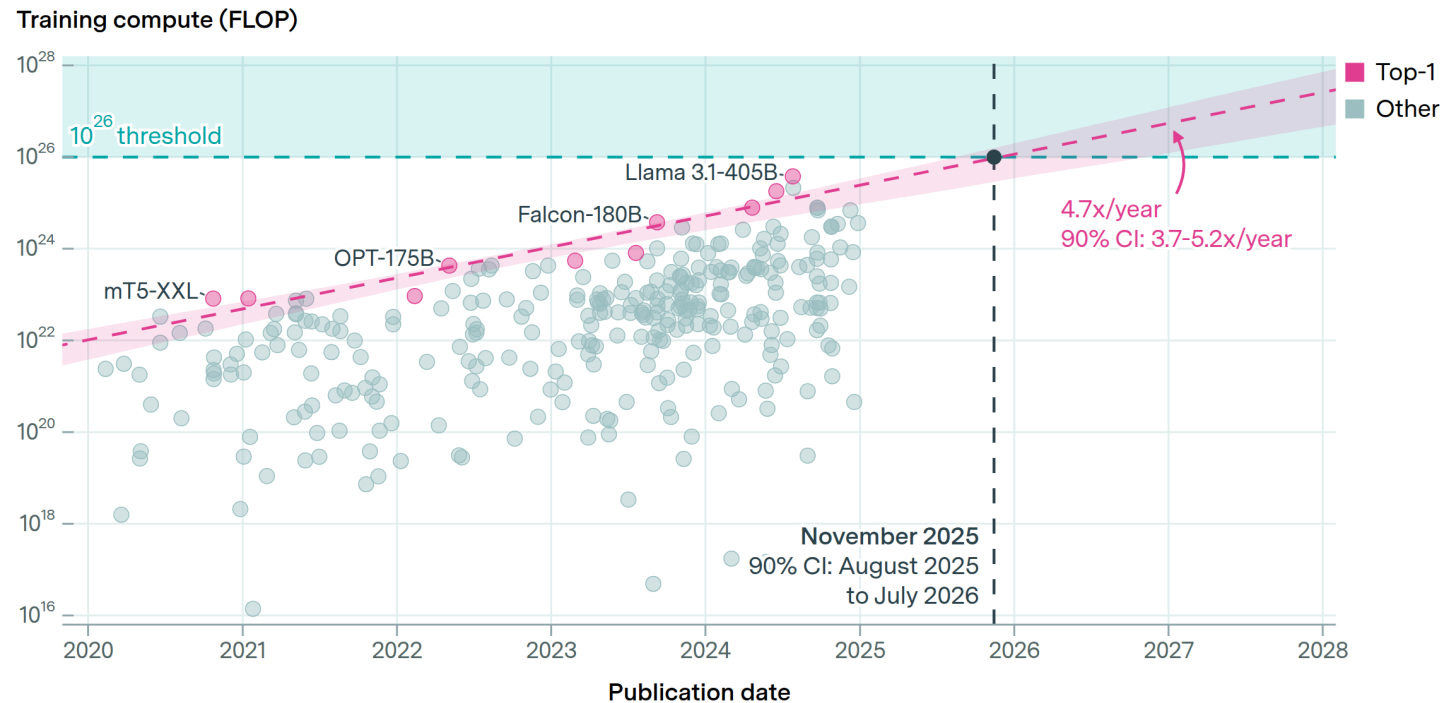


Q1 What does it take to **run** this, a robot or a coding agent that perceives, reasons, calls tools and acts in real time?

Q2 How to use AI to design chips? What evidence would convince you, what does it mean for how computers get **designed**?

AI Is the Defining Workload of Our Era

Training compute of notable open models



5x / year

frontier language-model training compute
since 2020 (Epoch AI): ~10,000x in six years

10²-10³ calls

model calls per agent task – inference, not
training, now dominates the bill

Every layer of the stack is being rebuilt
around this workload: **datacenters,**
GPUs, memory, interconnect, compilers,
runtimes.

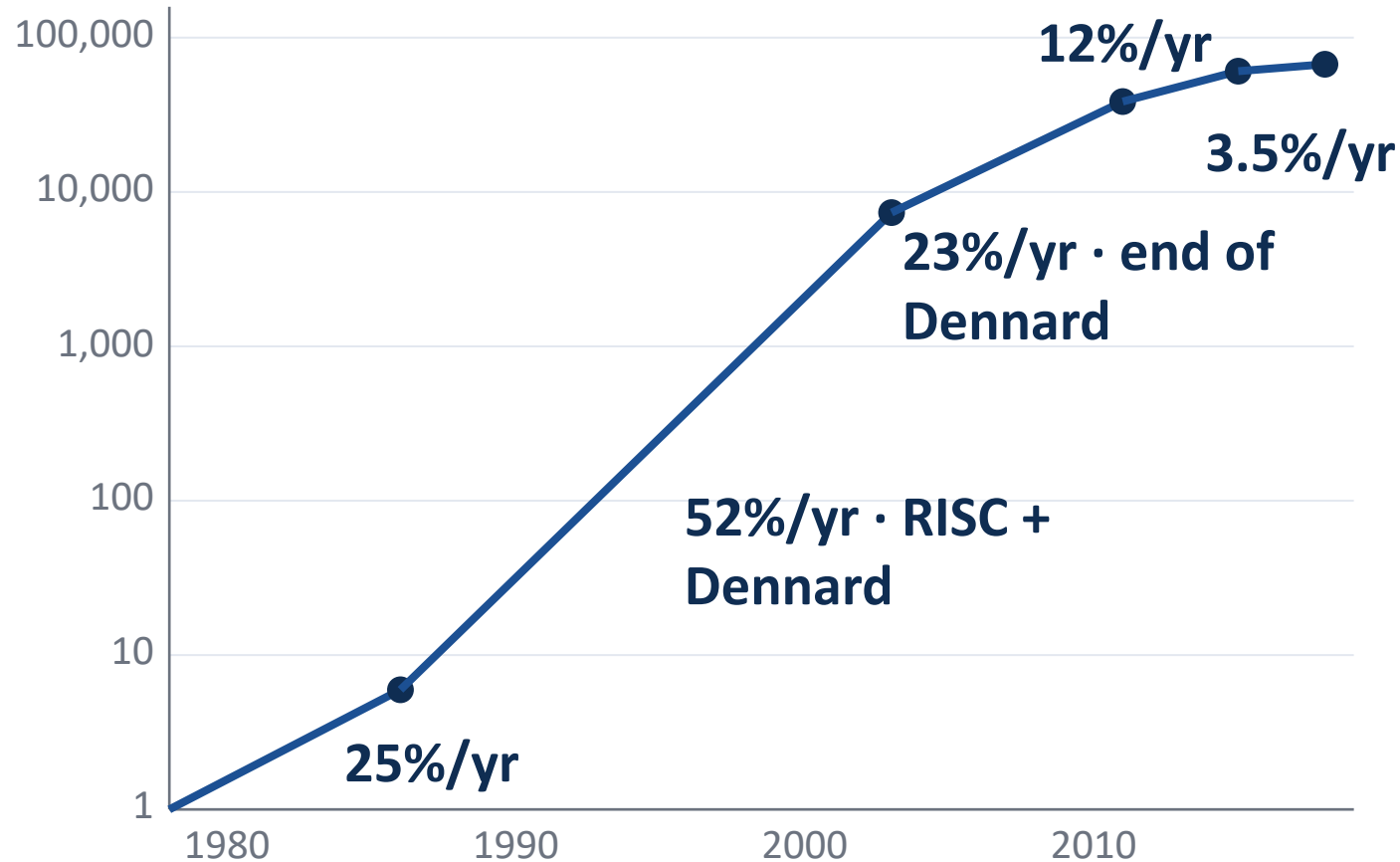
Sources: Epoch AI, Trends in AI (compute database); GPT-4 = Epoch estimate; Llama 3.1 = Meta



Training compute grows ~5x/year — inference adds a second, faster-growing demand

...While General-Purpose Hardware Has Stalled

Single-core performance relative to VAX-11/780 (log scale)



3.5% / year

single-core CPU performance, 2015-18 (H&P) → a doubling takes ~20 years

2.3 years

doubling time of peak FLOP/s per ML accelerator (Epoch AI) vs ~5 months for frontier training compute

“A new golden age for computer architecture”

— Hennessy & Patterson, CACM 2019

Curve redrawn from Hennessy & Patterson, CACM 62(2), Fig. 1; growth rates as reported

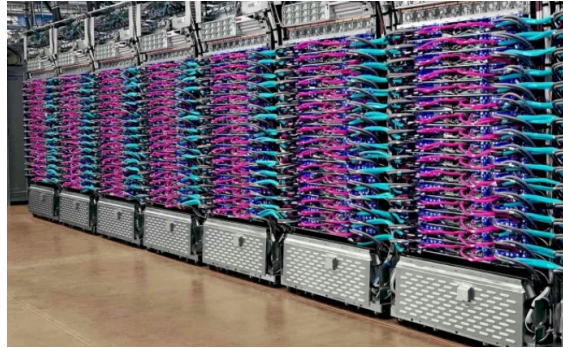


AI demand ×2 in 5 months, while accelerator FLOP/s ×2 in 2.3 years, CPU cores ×2 in 20 years

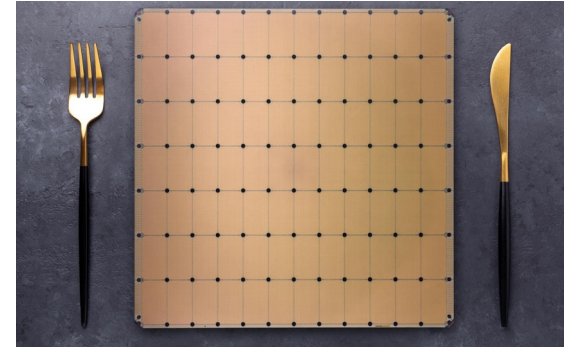
So Far, the Gap Was Closed by Specialization



Google TPU v4 pod



NVIDIA HGX SuperPOD



Cerebras wafer-scale



SambaNova DataScale

The compute-choice spectrum

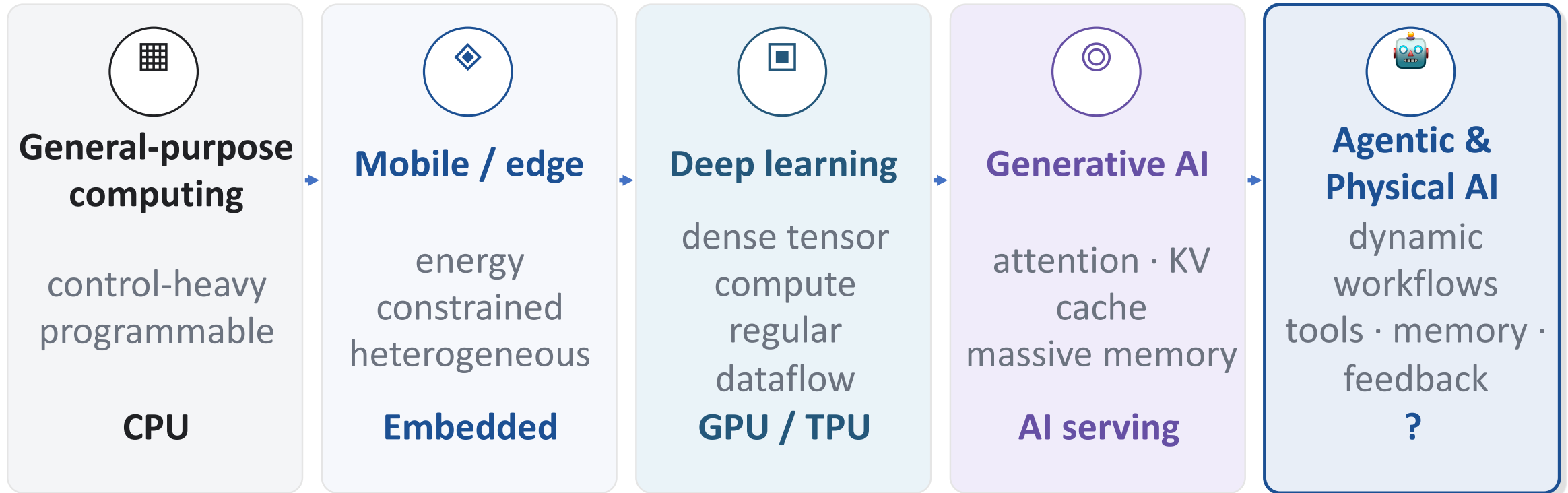


TPU v1 (2015): 15-30× the performance and 30-80× the perf/W of a contemporary CPU/GPU on inference (Jouppi + ISCA'17).

“Hardware and software must be co-designed to fully unleash the potential of AI.”

— Jensen Huang, NVIDIA

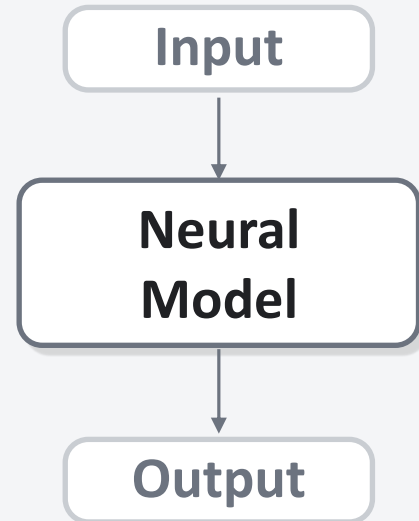
The Twist: The Workload Won't Sit Still



Every workload shift re-architected computing — we are inside the next one

AI Is Becoming a System, Not Just a Model

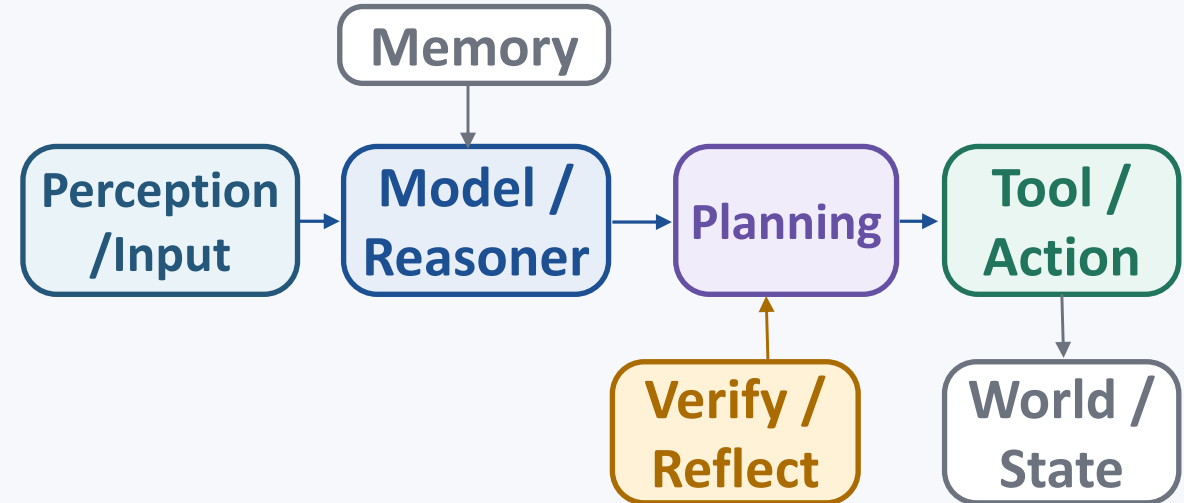
Yesterday: Model-centric AI



known graph · one model
accuracy / throughput



Today: AI as a dynamic workflow



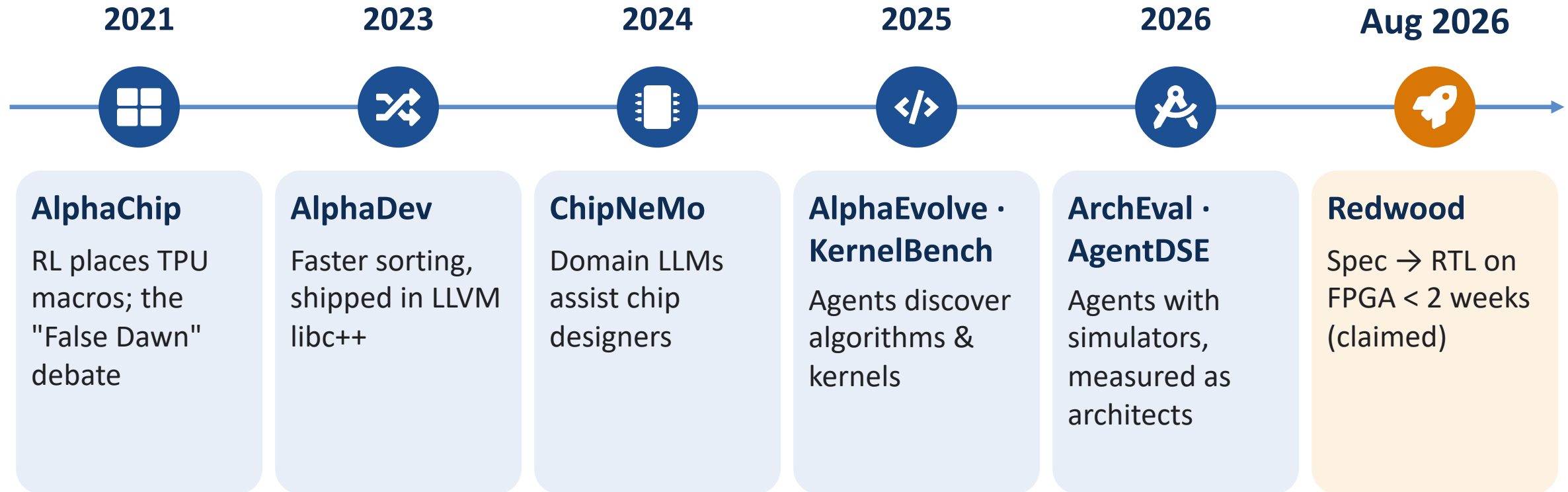
Dynamic Heterogeneous Stateful Interactive

The model is now one component inside a larger computation.



Dynamic, heterogeneous, interactive, stateful — it breaks our HW and serving assumptions

...and AI Is Starting to Design the Computer Itself



Every layer now has an "AI for X" literature — all six are on our reading list

Nature'21 (+ '24 addendum) · Nature'23 · NVIDIA'24 · DeepMind'25 · ICML'25 · arXiv'26 · Architect Labs'26

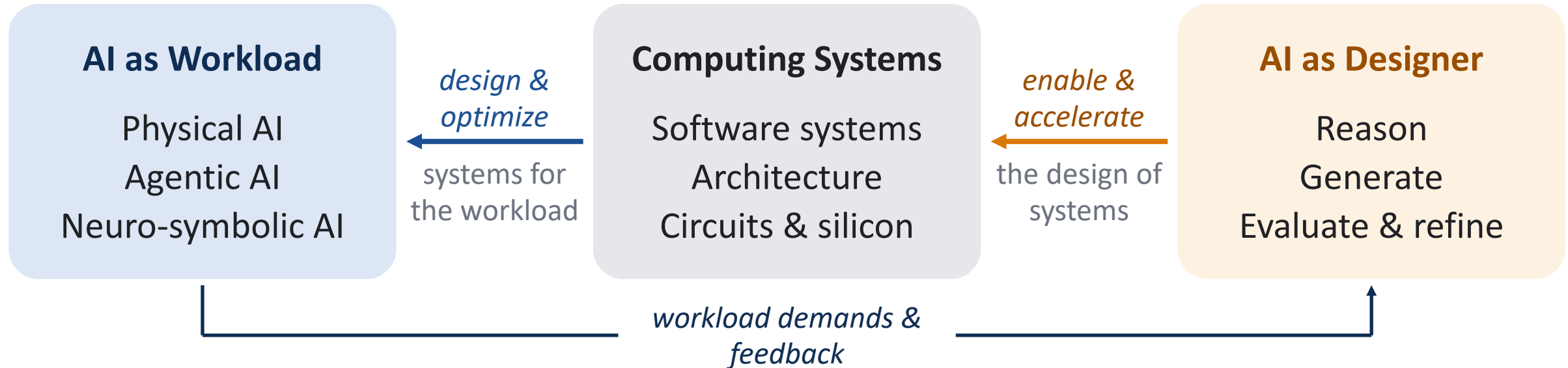


From ML as an optimizer in a loop to agents that propose, evaluate, realize

This Course: Computing for AI & AI for Computing

Computing for AI

AI for Computing



Course thesis: “Emerging AI workloads demand new hardware and system architectures, while AI is becoming a powerful tool for designing computing systems themselves.”



Module 1 (Wk 2-7): Computing for AI · Module 2 (Wk 8-12): AI for Computing

What You Will Learn



Profile & diagnose AI systems

Latency, throughput, utilization, energy, cost, find the bottleneck with roofline reasoning and traces

1



Reason across the stack

Model → software → runtime → architecture → memory → accelerator → SoC → deployment

2



Serve & accelerate new workloads

LLM serving, embodied-AI runtimes, neuro-symbolic acceleration, SoC design

3



Build AI that designs systems

Turn system and hardware design into an agent environment: state, actions, tools, feedback

4



Computer Architecture & ML Systems & VLSI

Understand the role of hardware acceleration in the AI boom;
Understand how to use AI for hardware and system design

5



Produce conference-style research

A scoped question, real baselines, experimental analysis, reproducible artifact

6



By end of semester: profile a system, propose cross-layer method, and defend it with evidence

What This Course Is — and Isn't



This course IS

- A research seminar, first offering, fast-moving field
- ~30 papers read critically; everyone leads one discussion
- A semester-long individual/team project on “computing for AI” or “AI for computing”
- An 8-10 page paper as a artifact
- Guest lectures from people building this today



It is NOT

- An introduction to deep learning or LLMs
- A CUDA, Verilog or EDA programming course
- Exams or problem sets, there are none
- A course you can take passively



Who should take it

Expected

- Systems / computer-organization basics
- Familiarity with ML concepts
- Programming + quantitative experiments

Helpful, not required

- CUDA, compilers, RTL / EDA, FPGA
- Robotics simulators, LLM agents
- Reading research papers

Expect (estimate): ~4 h/week of reading + 6-8 h/week of project work; you are expected to run experiments independently. Teams bring complementary skills, one or two helpful items each.

The Semester at a Glance

Module 1 · Computing for AI Weeks 2-7

Wk 1 · Sep 11

Course intro: AI as workload, AI as designer

[Survey form](#)

Wk 2 · Sep 18

Fundamentals: how to study an AI system; LLM inference

[Project Sel.](#)

Wk 3 · Sep 25

Serving systems for LLMs & agents

[Guest](#)

[Teams](#)

Wk 4 · Oct 2

AI hardware: accelerators & SoCs

[Guest](#)

[P0 due](#)

Wk 5 · Oct 9

Accelerating Physical AI I: inference & serving

[Guest](#)

Wk 6 · Oct 16

Accelerating Physical AI II: HW-SW co-design

[Guest](#)

[P1 due](#)

Wk 7 · Oct 23

Accelerating Neuro-symbolic & compositional AI

Module 2 · AI for Computing Weeks 8-12

Wk 9 mid-sem presentation · Wk 13 final posters · deadlines 11:59 PM ET

Wk 8 · Oct 30

AI for software: compilers & GPU kernels

[P2 due](#)

Wk 9 · Nov 6

Mid-semester project presentations

[Slides Nov 5](#)

Wk 10 · Nov 13

AI for architecture I: measuring & exploring

[Guest](#)

Wk 11 · Nov 20

AI for architecture II: agentic design

[Guest](#)

[P3 due](#)

Wk 12 · Dec 4

AI for RTL & chip physical design

[P4 due](#)

Wk 13 · Dec 11

Final poster session

[Final Poster](#)



13 Fridays · 9 seminar weeks · 6 guest lectures · 1 project, 6 checkpoints

Agenda for Today

- Course Overview
- **Course Logistics**
- Introduction to AI-Native Computing: HW for AI and AI for HW

2

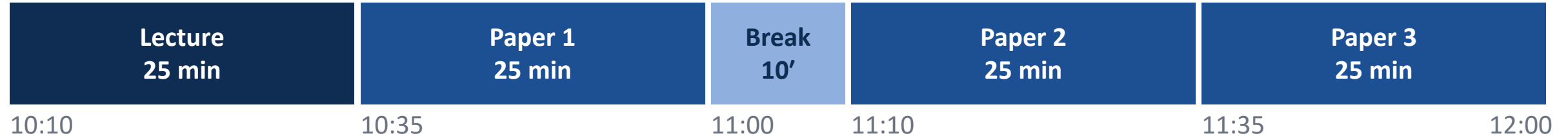
Part 2

How the Course Works

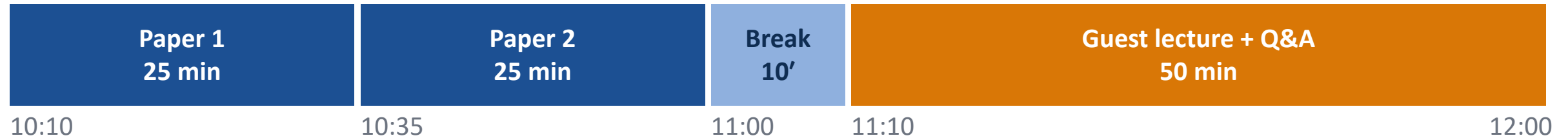
Format · leading a paper · grading · the project · policies · enrollment

A Typical Class Day

Regular seminar week



Guest-speaker weeks (Weeks 3-6, 10, 11)



Lecture (25 min)

Concepts, methods, connections across the papers.



Paper slots (25 min each)

Led by one of you: present, critique, run the discussion.



Guest lectures (50 min)

Builders of these systems, serving, accelerators, AI-assisted design



Week 9: midterm presentations · Week 13: final posters + best-project awards

Guest Speakers This Semester (Subject to update)



Jovan Stojkovic

UT Austin

*Serving systems for
LLMs & AI agents*

Wk 3



Tushar Krishna

Georgia Tech

*AI hardware:
accelerators & SoCs*

Wk 4



Yilun Du

Harvard

*Inference & serving for
embodied AI*

Wk 5



Luca Carloni

Columbia

*HW-SW co-design and
ESP Platform*

Wk 6



Sagar Karandikar

UC Berkeley

*AI for architecture:
measure & explore*

Wk 10



Vijay Janapa Reddi

Harvard

*Architecture 2.0 &
agentic design*

Wk 11



Guest weeks: two papers before the break, then a 50-minute lecture + Q&A

Leading a Paper: The 25-Minute Format

12 min Teach it

problem · context · mechanism · the minimum results needed

8 min Take it apart

claims · baselines · assumptions · missing evidence

5 min Run the room

discussion around 2-3 precise questions



If you are presenting

- Read the full paper, appendices, and artifact docs
- Slides due Thursday 11:59 PM ET (the night before)
- Your critique counts as much as your summary - 15%



Everyone else

- Read the full paper
- Arrive with at least one discussion question, that is your participation (10%)

9 → 23

seminar weeks →
paper slots

1 each

everyone leads once; ~7
co-led if 30 enroll



Preference form due Fri Sep 18, 11:59 PM

Pick the paper you want to present

How We Read: Evidence-Centered Discussion

1 What is the exact central claim?

The one sentence the paper must defend.

2 Which experiment most directly supports it?

Find the figure or table that carries the weight. Is it the one the paper leads with?

3 Which hidden assumption is most likely to break?

Workload mix, scale, hardware generation, oracle information, a fixed model.

4 Is the baseline fair and budget-matched?

Same tuning effort, compute, tokens and wall-clock, including classical, non-agentic baselines.

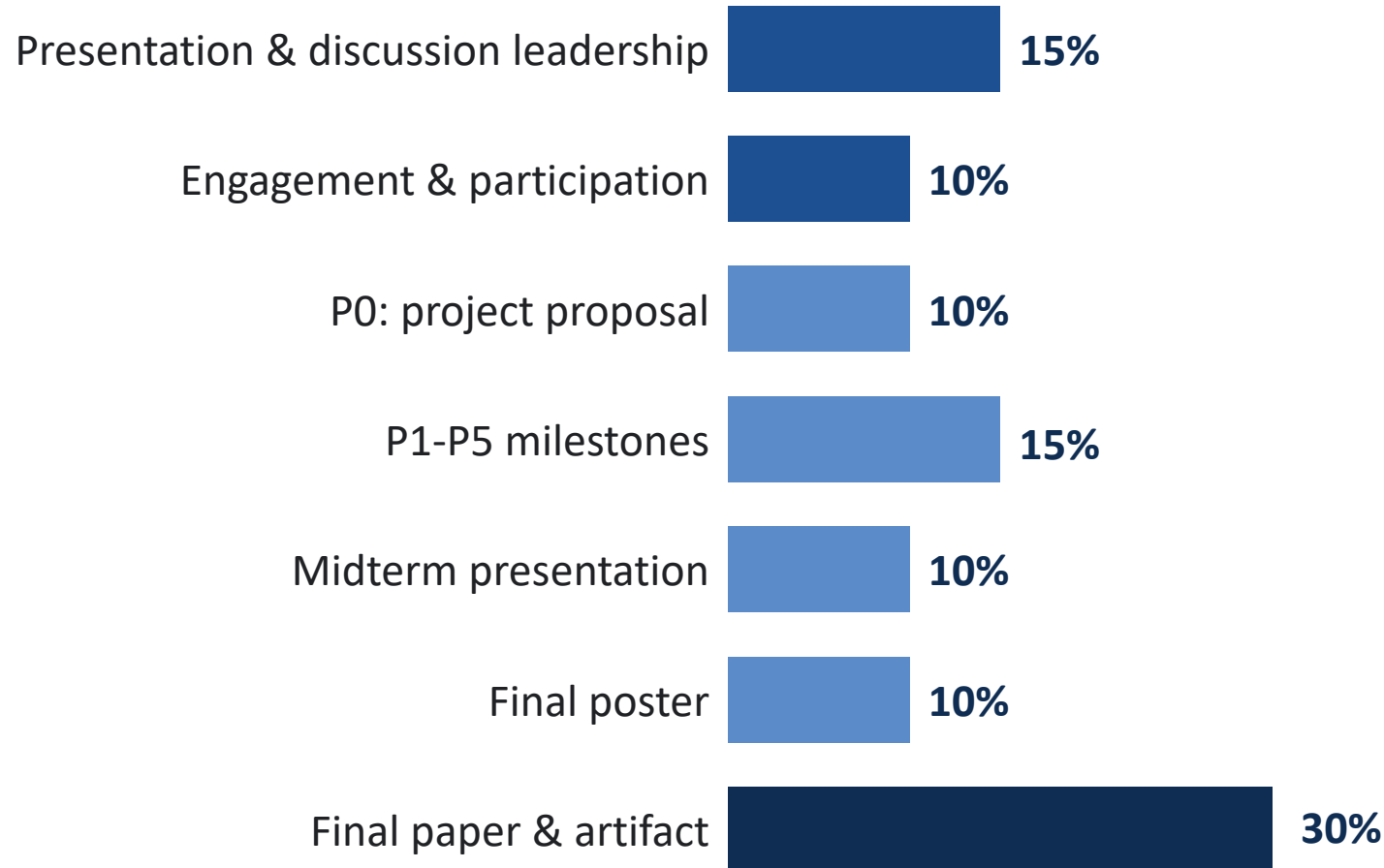
5 Which conclusion extends beyond the evidence?

Where does “we show” quietly become “this implies”?

6 What would change your confidence?

If you could run one thing, what would falsify the claim?

Grading



What grades reflect

- Research judgment
- Technical execution
- Evidence quality
- Communication
- Reproducibility

Seminar 25% · **Project 75%** · no exams, no problem sets



75% of the grade is the project; 40% is the final paper & poster — start early

The Project: A Semester-Long Research Effort



Track A · Computing for AI

- Profile, serve, schedule, map, accelerate or make reliable an emerging workload
- LLM inference, agentic pipelines, physical AI / VLAs / world models, neuro-symbolic reasoning
- Outputs: a runtime, scheduler, mapping, accelerator, or benchmark + analysis



Track B · AI for Computing

- Build and rigorously evaluate agents that design or optimize systems
- Software optimization, compilers, GPU kernels, architecture DSE, RTL / EDA, verification
- Outputs: agent + environment, benchmark, evaluation vs classical & non-agentic baselines



Project candidates

Introduced in Week 2 (Sep 18); your own proposals welcome



Bid

Bidding form due Wed Sep 23, rank directions



Teams of 1-3

Formed by Fri Sep 25; complementary skills over shared interests



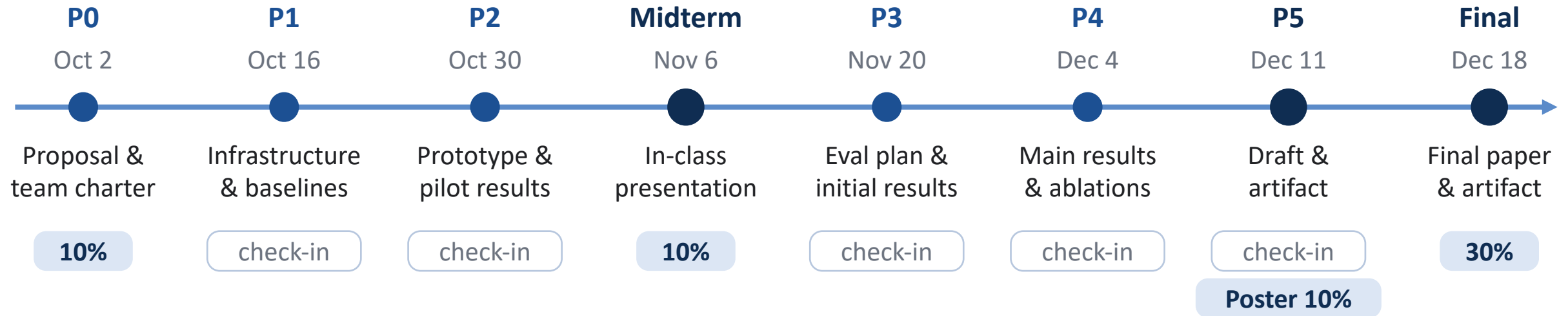
P0 proposal

Due Fri Oct 2: question, claim, baselines, fallback



Every project must reach a clear, verifiable conclusion — strong ones grow into papers

Project Milestones



All written deliverables 11:59 PM ET · P1-P5 together = 15%, graded on completeness · nothing due at Thanksgiving

The final submission (Dec 18) contains

-  8-10 page conference-style paper
-  Introduction, background, related work, proposed methodology, evaluation, future work, conclusion
-  Repo with environment + smoke test
-  Experiment manifest: seeds, configs, machines

 **Conduct a semester-long conference/journal publication-level research project.**

Course Policies



AI use: encouraged (and required)

- Encouraged for brainstorming, literature search, coding, debugging, writing
- Every citation checked at the source, every number traces to an experiment. Agent output is not evidence by itself
- Disclose how you use AI during the project
- **During project Q&A, you are expected to present technical details and answer questions clearly w/o referring to AI tools**



Academic integrity

- Columbia's academic-integrity policies apply
- Serious violations: fabricated citations, invented experiments, altered logs, undisclosed result selection



Late work

- One 48-hour grace pass per team on a written milestone, request it before the deadline
- Not for presentations, the poster, or the final submission



Use AI actively and creatively. Learning to collaborate effectively with AI tools is part of this course: explore more ideas, build faster, and deepen your understanding.

Prerequisites & Expectations

What I assume you bring

Expected

Computer architecture & system basics:
memory hierarchy, parallelism, profiling
ML + programming: training vs. inference

Helpful, not required

CUDA · compilers · RTL / EDA · simulators ·
FPGA · LLM agents

Project readiness

Teams of 1-3, pick a partner who covers what
you don't

What I expect from you, every week

 **Read the core of 2-3 papers**
abstract, intro, method, key results, limitations —
about 3-4 h

 **Show up with a question**
one question per paper = participation grade
(10%)

 **Lead one paper, once**
25 min; slides due Thu 11:59 PM ET; critique
graded (15%)

 **Conduct project in a weekly progress**
6-8 h/week; run experiments independently on
hardware or a simulator you can access

 **No one covers the whole stack — bring complementary skills; read critically, measure carefully**

Enrollment, Slack, and Your To-Do List

Enrollment (cap: 30)

- **Today, 11:40**
Background survey in class
- **Sat Sep 12, 11:59 PM**
Survey deadline if not finished
- **Mon Sep 14, morning**
Instructor manually enrolls students
- **Fri Sep 18**
Last day to add · Week 2 class

Communication

- <https://wan-research-group.github.io/coms6998-f26/>
-  **Canvas: announcements & submissions**
Milestones, slides, final artifact
-  **Email: private matters**
zishen.wan@columbia.edu
-  **Office hours**
After class &
Wed 5-6 PM, CSB 522

Before Fri Sep 18

- ☐ **Background survey**
today; Sat 11:59 PM
- ☐ **Read for Week 2**
Roofline · Tail · FlashAttn
- ☐ **Paper preference form**
due Fri Sep 18, 11:59 PM
- ☐ **Project bidding form**
due Wed Sep 23
- ☐ **Optional reading**
Bitter Lesson · Arch 2.0

 **Survey by Saturday night → enrollment Monday → Week 2 on Sep 18 (background & project candidates)**

Agenda for Today

- Course Overview
- Course Logistics
- **Introduction to AI-Native Computing: HW for AI and AI for HW**



Break

Back at 11:00 — Part 3: the lecture

3

Part 3 · Lecture

AI-Native Computing

AI as Workload, AI as Designer — and why they are one loop

The Loop of Computing and AI



Act 1

Computing for AI

What is “the workload” now, and where does the bottleneck go?

Three workload classes · one serving case you decide · three systems we built for them · the bottleneck moves across the stack



Act 2

AI for Computing

Can AI design the computing systems (software, computer architecture, chips), and how would we know?

The design loop · a ladder from optimizer to designer · Redwood, critically · the evaluator bottleneck



Act 3

The Loop

How do the two directions feed each other?

The AI-native computing loop

The Loop of Computing and AI



Act 1

Computing for AI

What is “the workload” now, and where does the bottleneck go?

Three workload classes · one serving case you decide · three systems we built for them · the bottleneck moves across the stack



Act 2

AI for Computing

Can AI design the computing systems (software, computer architecture, chips), and how would we know?

The design loop · a ladder from optimizer to designer · Redwood, critically · the evaluator bottleneck



Act 3

The Loop

How do the two directions feed each other?

The AI-native computing loop

What Is “the Workload” Now?



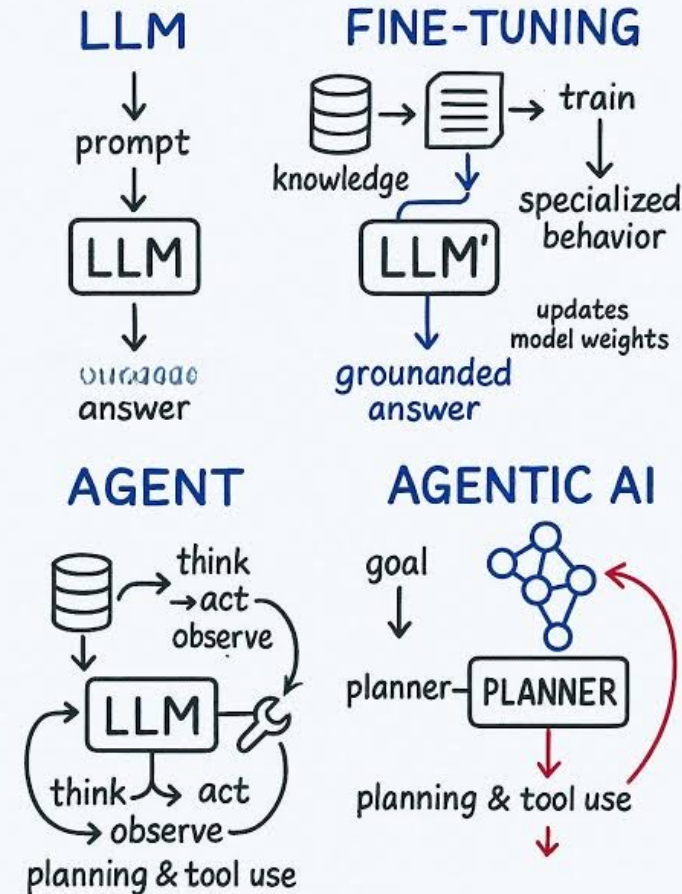
LLM & agentic AI

Autoregressive generation wrapped in workflows of model calls, tools, memory and verification.

What is new for systems

- Decode is memory-bound; the KV cache is state
- The call graph is unknown until runtime
- Quality is a knob: model size, verification

Weeks 2-3



What Is “the Workload” Now?



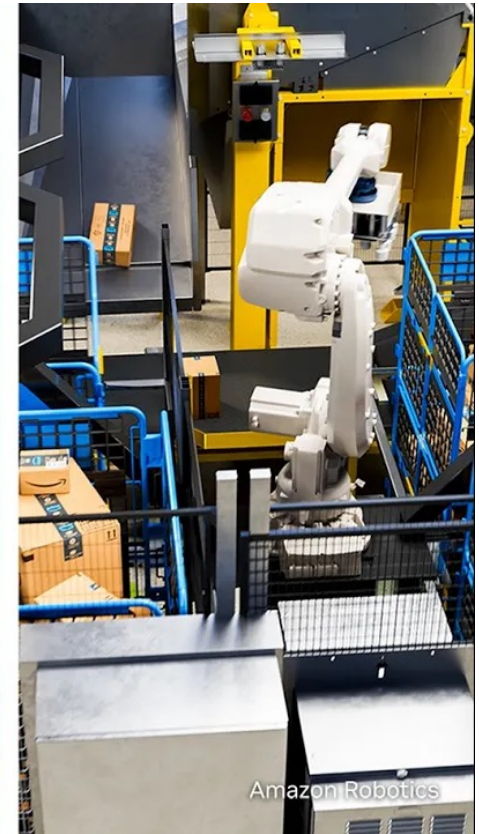
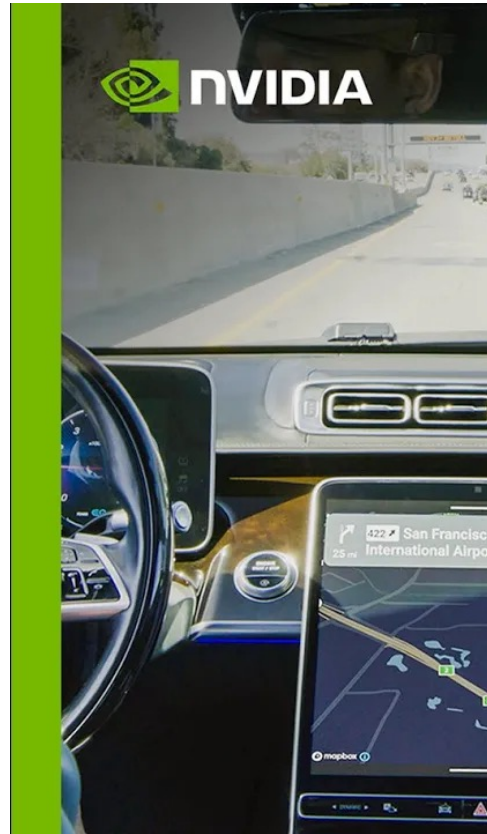
Physical AI

Perceive-reason-act loops on robots, vehicles and XR devices: VLAs and LLM planners on classical autonomy.

What is new for systems

- Hard real-time: control loops at 100-1000 Hz
- Battery, thermal and weight budgets
- Safety: a late answer is a wrong answer

Weeks 5-6



What Is “the Workload” Now?



Neuro-symbolic & compositional AI

Neural perception coupled with symbolic logic, probabilistic reasoning, planners and verifiers.

What is new for systems

- Irregular, control-heavy kernels
- Low arithmetic intensity, GPUs idle
- Tight neuro ↔ symbolic coupling

Weeks 7

Symbolic vs Neural vs Neuro-symbolic AI

Symbolic AI



- Logic and rules
- Human interpretable
- Difficult to scale

Neural Networks



- Pattern recognition
- Learns from data
- Data hungry

Neuro-symbolic AI



- Learning and reasoning
- Combines knowledge
- More robust

What Is “the Workload” Now?



LLM & agentic AI

Autoregressive generation wrapped in workflows of model calls, tools, memory and verification.

What is new for systems

- Decode is memory-bound; the KV cache is state
- The call graph is unknown until runtime
- Quality is a knob: model size, verification

Weeks 2-3



Physical AI

Perceive-reason-act loops on robots, vehicles and XR devices: VLAs and LLM planners on classical autonomy.

What is new for systems

- Hard real-time: control loops at 100-1000 Hz
- Battery, thermal and weight budgets
- Safety: a late answer is a wrong answer

Weeks 5-6



Neuro-symbolic & compositional AI

Neural perception coupled with symbolic logic, probabilistic reasoning, planners and verifiers.

What is new for systems

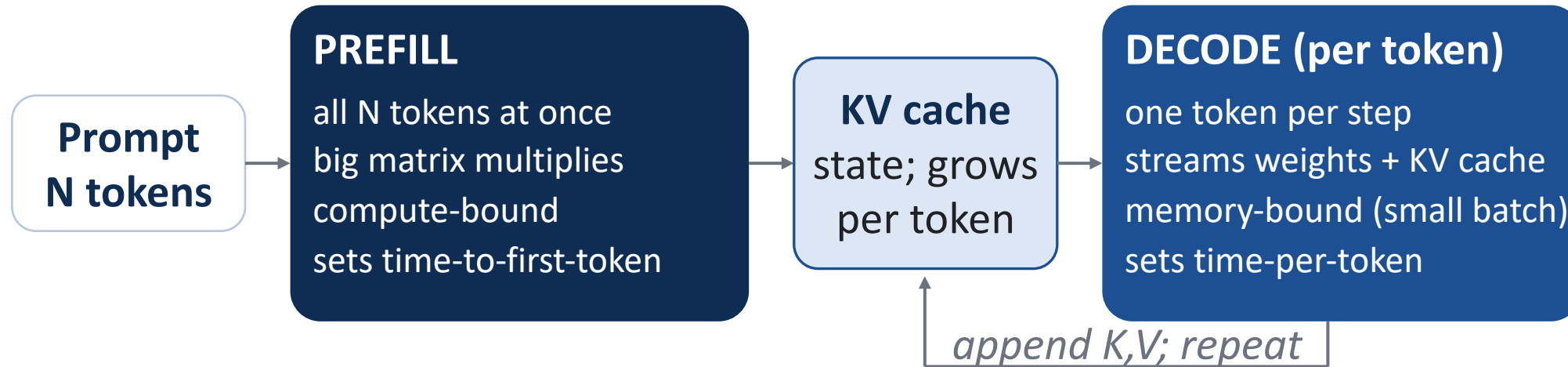
- Irregular, control-heavy kernels
- Low arithmetic intensity, GPUs idle
- Tight neuro ↔ symbolic coupling

Week 7



Common thread: not one model call — workflows and loops with state, deadlines, feedback

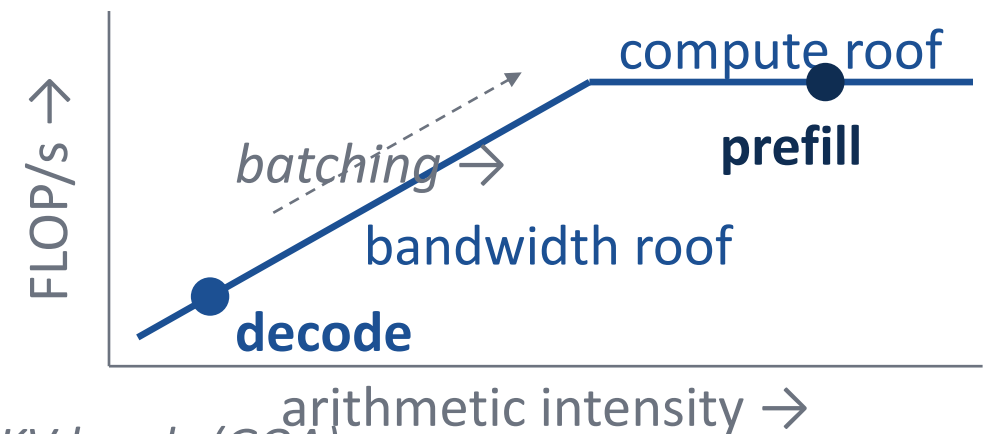
Workload 1: LLM Inference Is Two Different Programs



Back-of-envelope: Llama-3-70B FP16 $\approx$ 140 GB

- Capacity wall: 140 GB > 80 GB H100 $\rightarrow \geq 2$ GPUs
- Bandwidth wall: ~ 70 GB/GPU per step ≈ 21 ms $\rightarrow \leq 48$ tok/s
- KV cache ≈ 320 KB/token $\rightarrow$ 32k context ≈ 10 GB

The two phases on a roofline (schematic)



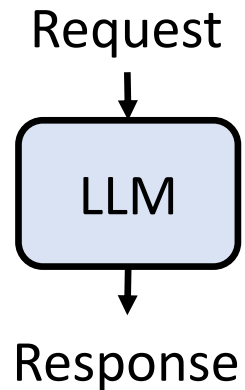
H100 SXM: 80 GB HBM3, 3.35 TB/s; Llama-3-70B: 80 layers, 8 KV heads (GQA)



More FLOPs alone don't speed up inference — capacity and data movement set the limit

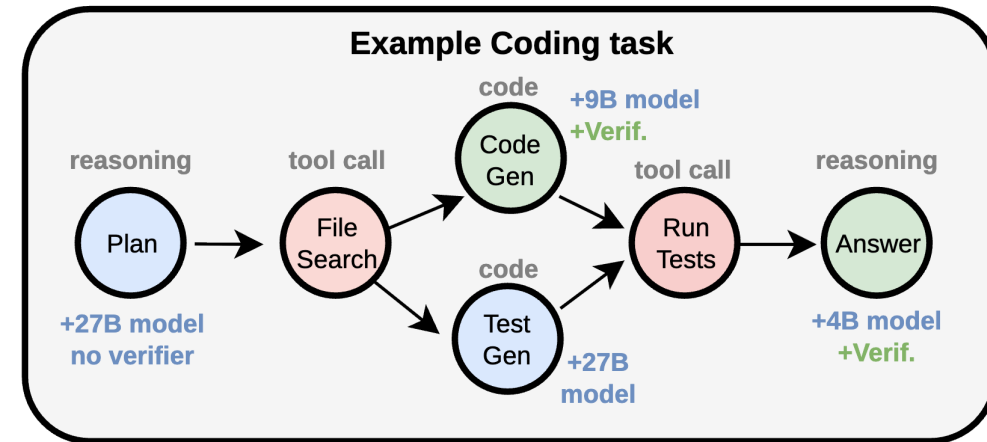
...and Agents Turn One Call into a Workflow

Traditional Model Serving



Unit of optimization: one model call

Agentic AI



Who decides how this workflow should actually run?

Missing control boundary

Orchestrator

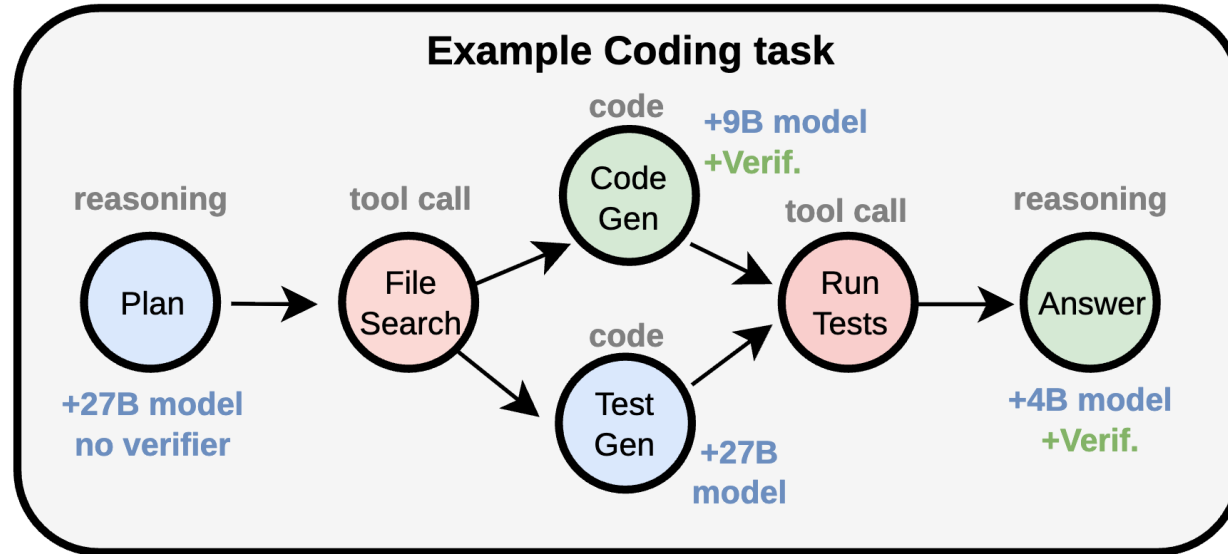
Sees the DAG
Knows dependencies
No live GPU / fleet cost

Model Serving
Engine

Sees queues/GPU state,
Optimizes batching & KV cache
No workflow semantics

Agentic serving ≠ Serving a sequence of independent LLM calls

Where Would You Spend the Stronger Model?



Coding-agent workflow

? Your call

You can afford a 27B model on only **some** nodes and a 4B model elsewhere, plus a verifier that re-checks a node's output.

Which nodes get the strong model? Where does a verifier pay off? Why?

⚡ Plan A · cheap

4B everywhere, no verifier

🏆 Plan B · expensive

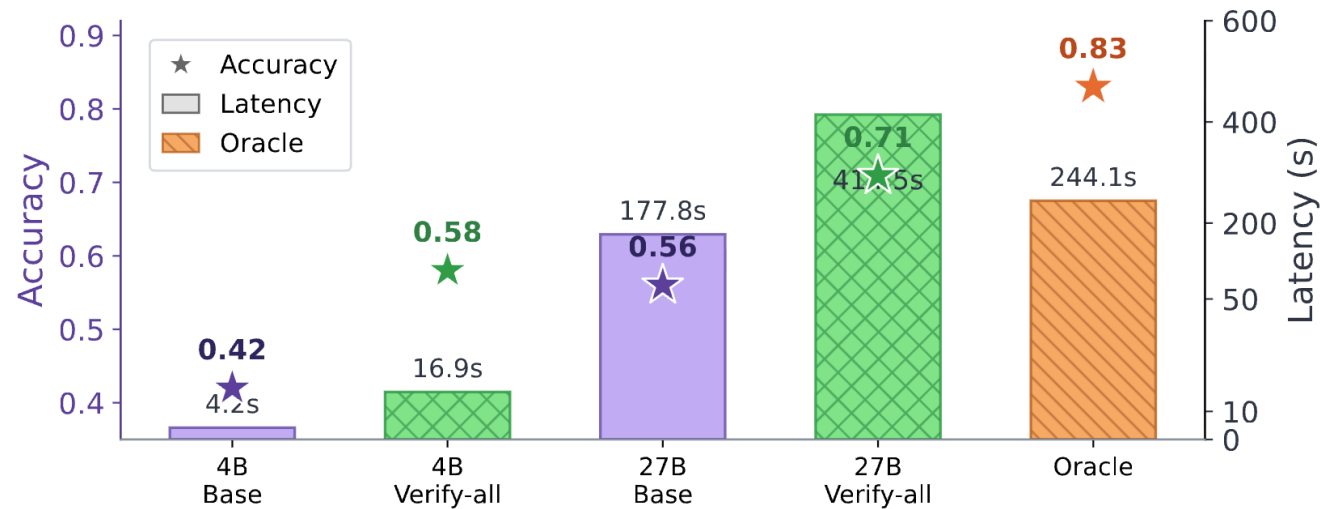
27B everywhere (+ verifiers)

⚡ Plan C · mixed

strong model where errors propagate; verify the rest

Which plan will you vote?

What Verification Bought, and What It Cost



Coding-agent benchmark, same workflow, five execution plans.
 Stars: accuracy · bars: end-to-end latency (log scale).
 Oracle = perfect per-task routing, an upper bound, not deployable.

Plan	Accuracy	Latency
4B + verify every node	0.58	16.9 s

27B, no verifier
≈ 10.5× faster

at the same accuracy (0.58 vs 0.56 is within noise on this benchmark), a small model plus a verifier matched a model 7× its size



What did verification buy us, and when would this trade-off stop working?

Think: tasks without a cheap checker, verifier false positives, load on the 27B backend, nodes where errors propagate.

Qian, Wan, et al., DyServe, arXiv 2607.02942 (2026), one example; Week 3 reads Parrot, Agentix, DistServe



The unit of optimization is the whole task: quality, latency and cost trade against each other per node, not per call

Workload 2: Physical AI Computes a Closed Loop



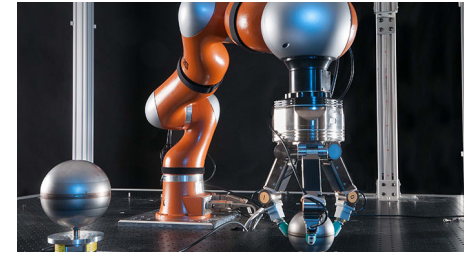
humanoids



autonomous driving



drones



manipulation



AR / VR

Two timescales

Cognition

LLM/VLM planning ·
~0.1-1 Hz



Autonomy

perception 10-60 Hz ·
control 100-1000 Hz

Sensing & Perception

cameras, LiDAR, IMU
→ state



Reasoning & Planning

VLA / LLM planner,
world model



Control & Action

trajectory, actuation,
safety



*the environment answers back — every action
changes the next input*

Deadlines are physical: 100 ms at 10 m/s = 1 m of travel — and it all runs on a battery



If inference gets 2× faster, does the robot finish 2× faster? Why not?

Why Physical AI Is Hard for Computing

Physical AI

Cognition



Autonomy

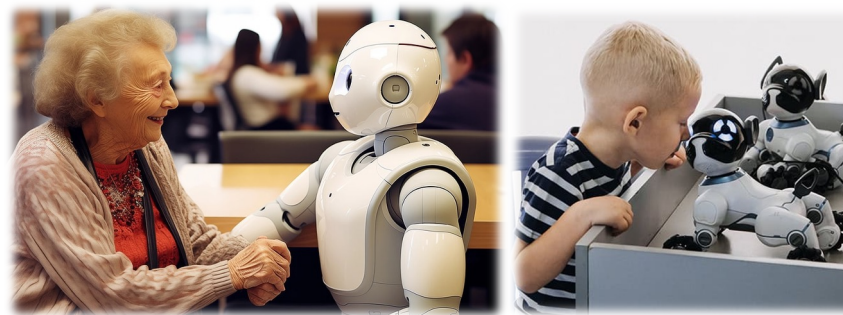


Tight coupling
across timescales

Real-Time Performance

	Goal	Current
Cognition Loop:	>1 Hz	<0.1 Hz
Autonomy Loop:	>100 Hz	<1 Hz
		✗

Critical Safety & Reliability



Strict Energy Budgets



Battery:

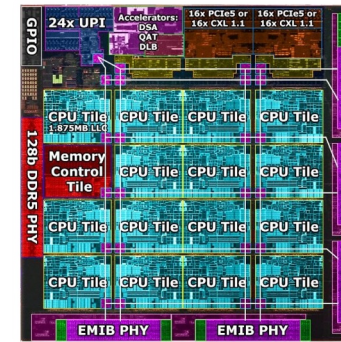
Battery Capacity = 64 Wh

Average Runtime = 20 mins

Standby Time = 40 mins

Recharge Time = 13 mins

Complex Design Space



Design Space:

Algorithm: $O(10^{11})$

Hardware: $O(10^8)$

Dynamics: $O(10^3)$

Sensors: $O(10^2)$



Real-time · energy · safety · astronomical design space — gaps are often 10-100×

Example: Co-Design SLAM Hardware for a Nano-Drone



Localization on board a nano-drone, the loop must close in $< 20\text{ ms}$ on $< 2\text{ W}$

Platform	Latency	Power	Verdict
Goal	$< 20\text{ ms}$	$< 2\text{ W}$	<i>both at once</i>
Desktop CPU	80 ms	50 W	too much power
Embedded CPU	500 ms	3 W	too slow
Co-designed FPGA	10 ms	1.5 W	meets both



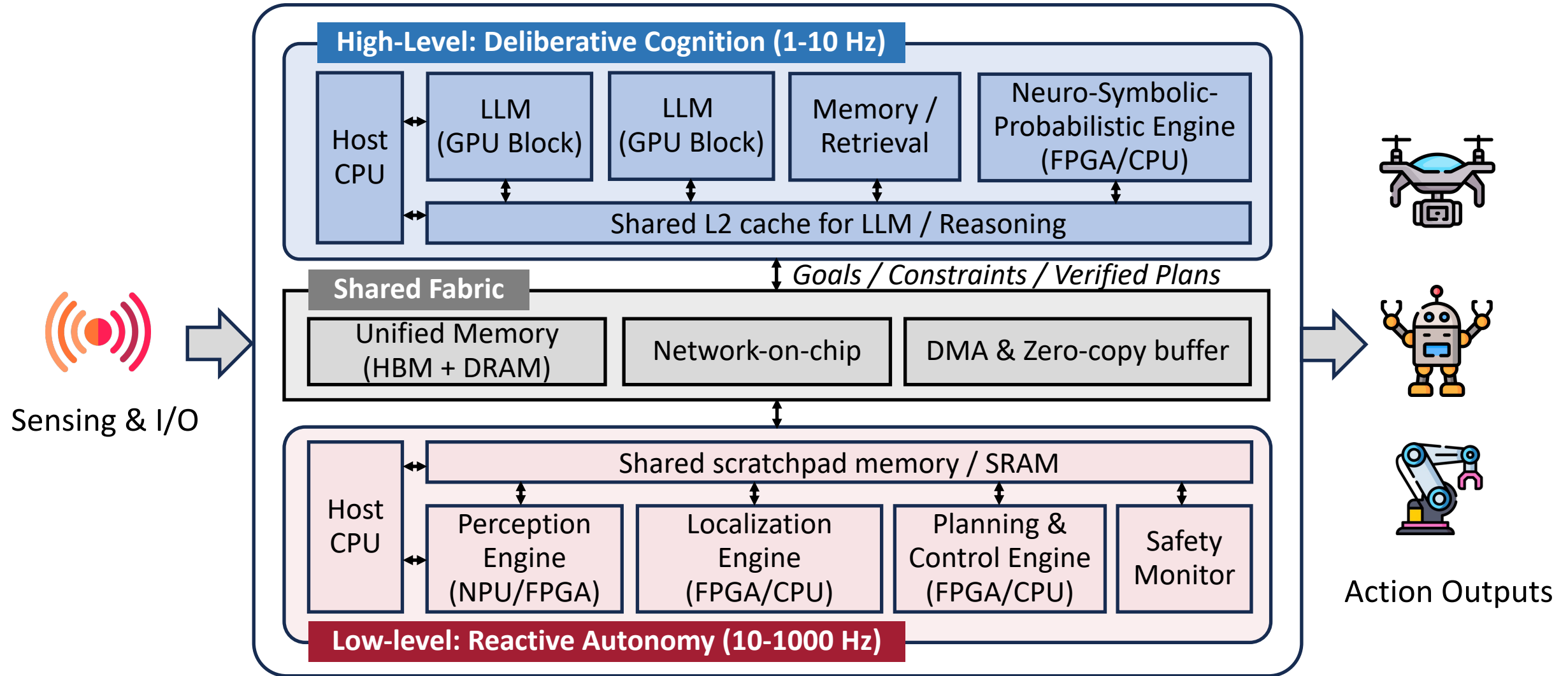
Measure: ~95% of the time is the back-end solver, consisting of small, dependent, irregular matrix ops



Co-design: keep matrix diagonal, $O(n^3) \rightarrow O(n)$; exploit symmetry + sparsity, leading to 4.1× less memory, 2.7 MB fits on chip

 Software-hardware co-design improves physical AI system performance and efficiency

ReCA: SoC Bridges Cognition and Autonomy

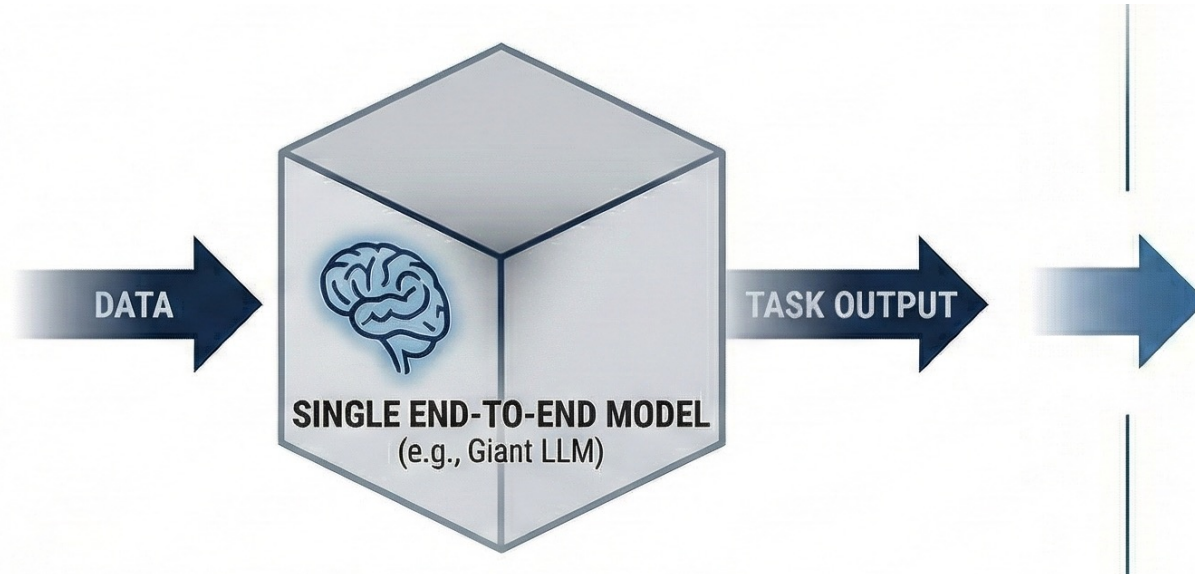


Wan et al., ReCA: Integrated Acceleration for Embodied Agent Planning, ASPLOS 2025

➡ **Heterogenous SoC bridges reactive autonomy and deliberative cognition for physical agents**

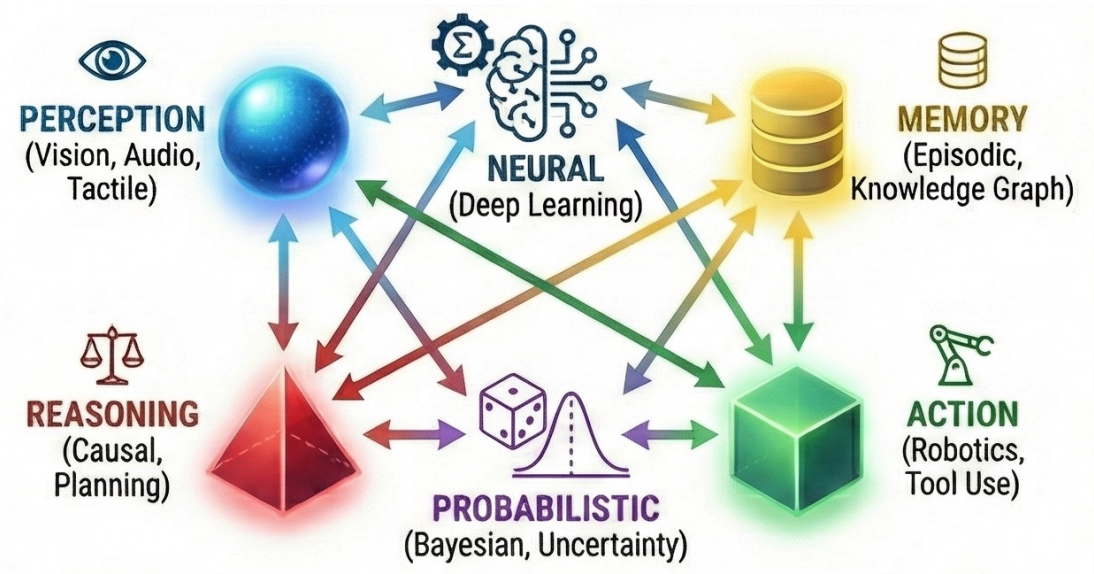
Workload 3: Compositional AI

Neuro-only



- Opaque end-to-end execution
- Limited verifiability and robustness
- No explicit physical grounding
- Poor generalization under shift

Compositional



- Interpretable, modular structure
- Verifiable interfaces by design
- Can be grounded in physical state
- Often more data-efficient



Structure gives you checkable interfaces, not automatic trust or generalization

...but Symbolic Kernels Hit a Wall on Today's Hardware



An LLM decomposes the task; a symbolic planner checks feasibility

> 20 s of symbolic planning per task — the LLM is not the consistent bottleneck

Why the GPU idles

GPU profile	Neural MatMul	Symbolic kernels
ALU utilization	98%	29-53%
Warp efficiency	96%	49-59%
Eligible warps / cycle	7.2	2.1-2.8

Not bugs — algorithmic: irregular graphs, branching, pointer chasing

↔ **The obvious fix does not rescue it.** CPU for symbolic, GPU for neural: symbolic stays slow on the CPU, and fine-grained coupling turns every hand-off into a PCIe transfer

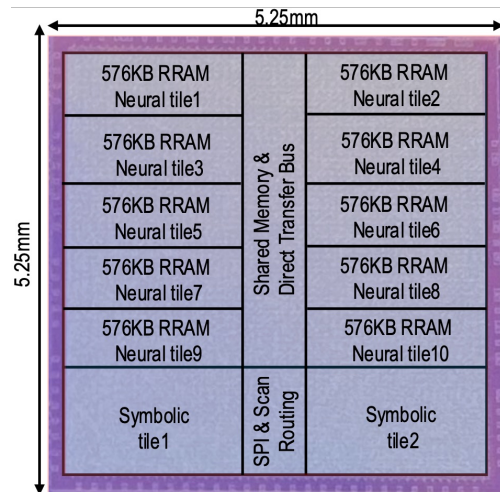
[Wan, et al, ASPLOS 2026] [Wan, et al., HPCA 2025]

⚠ **Symbolic kernels crawl on GPUs: irregular graphs, heavy control flow, little parallelism**

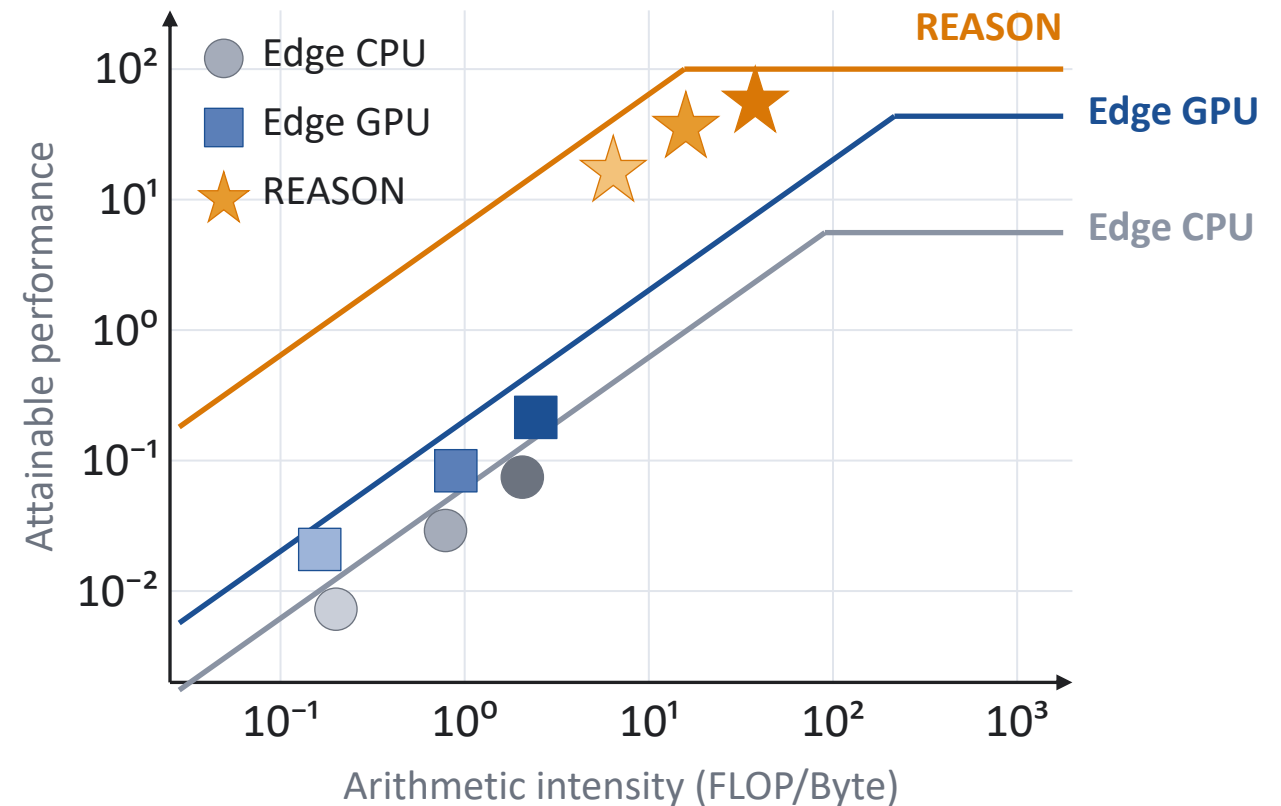
Example: REASON – From Reasoning Graphs to a SoC

Heterogenous SoC

- IR: one DAG for every reasoner
- PE reconfigures: logic · prob · sparse
- RRAM + SRAM, RISC-V cores
- Idle units powered off between phases



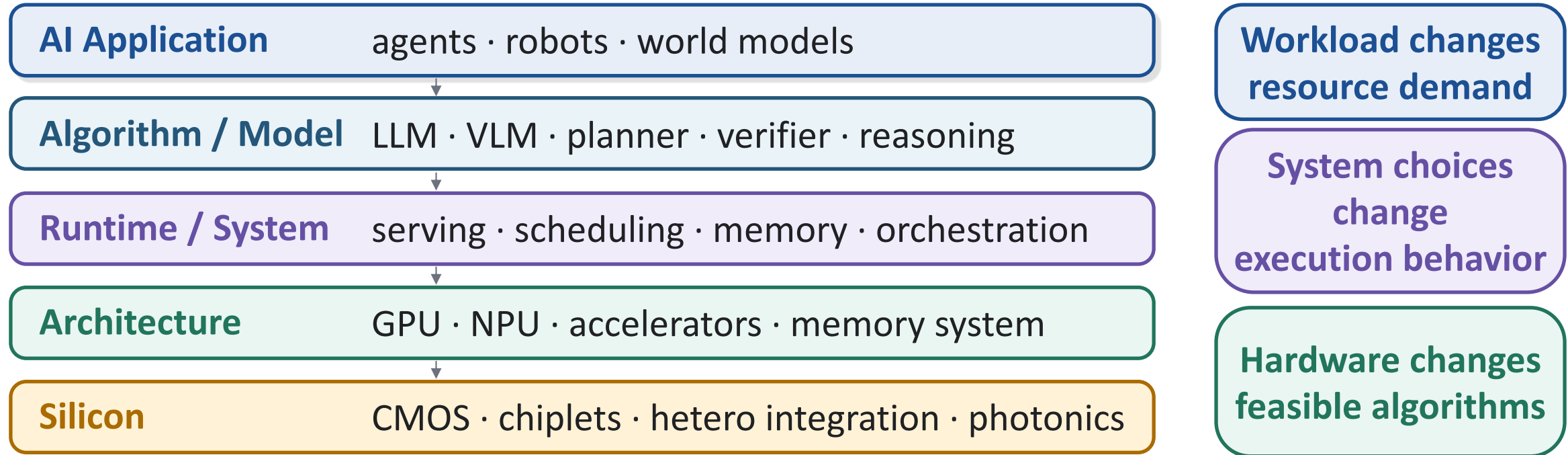
Measured
 40 nm · 27.6 mm²
0.67 W vs 15 W (TX2)
1.9-6.5× less power
 w/ power mgmt



[Wan, et al, HPCA 2026] [Liu*, Wan*, et al., JSSC 2026]

On REASON the same kernels climb 2-3 orders of magnitude to the roof

The Bottleneck Moves Across the Entire Stack



Co-design: “changing the interface or contract between areas allows better solutions than work in a single area.”



Within a layer, trade-offs are zero-sum; across layers you exploit each layer’s don’t-cares. -- S. Subramanian (Google)



No single layer fixes it — co-design across algorithms, systems, architecture, silicon

Before You Build: Measure

Task quality

accuracy, pass@k,
success rate — the
number a systems
paper hides

Latency

p50 / p99 tail, not the
mean; TTFT & TPOT;
loop period

Goodput

requests or tokens/s
that meet the SLO —
not raw throughput

Utilization

MFU / MBU; where
on the roofline; idle
time in traces

Energy & power

J/token, J/task, W
under a thermal cap;
battery life

Cost

\$/token, \$/task, GPU-
hours, API tokens —
with quality

Finding the bottleneck

- **Roofline reasoning**
compute- or bandwidth-bound?
- **Queueing & tails**
why p99 dominates loops
- **Tracing & profiling**
Nsight · torch.profiler · ROS
- **Benchmarks**
MLPerf · RobotPerf · KernelBench
- **Analytical models**
VLA-Perf · Concorde · Timeloop

Project rule: quality and systems metrics jointly, with budgets and a held-out condition

 **If we cannot measure, we cannot optimize**

Module 1 Takeaways



The unit of computation is moving

From one model call to a workflow, a control loop, a reasoning graph — with state, deadlines and feedback.



The bottleneck is cross-layer

Decode bandwidth, KV memory, orchestration gaps, real-time control, irregular kernels.



Measure jointly, then build

Quality × tail latency × energy × cost — with roofline, tracing and benchmarks, before proposing hardware.

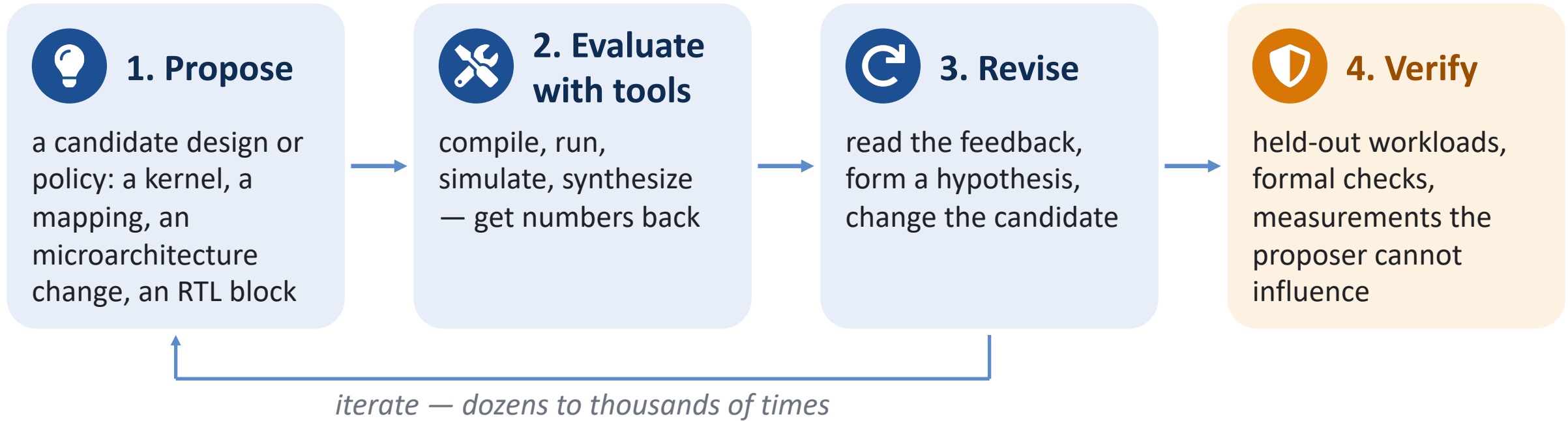
Module 1 · Weeks 2-7:

fundamentals → serving → accelerators & SoCs → physical AI → compositional AI



Module 2 flips the direction: can the hardest workload also be the best designer?

Flip the Direction: AI as Designer



 This is the loop a human architect runs. **If AI is the hardest workload we have had, can it also run this loop better than we do?** Every step is a research question: what to propose, how fast the tools answer, how to trust the result.

 **AI for Computing: the stack becomes the object AI proposes, evaluates, verifies**

Why AI for Computing, and Why Now



Design spaces exploded

- Mappings, SoC configs, placements: 10^8 - 10^{11} candidates
- Cycle-level simulators: 10^3 - $10^5\times$ slower than real time
- Nobody sweeps that by hand



Timescales don't match

- A chip generation: 2-4 years, hundreds of millions of dollars
- Frontier models: a new architecture every few months
- Design under uncertainty: “the hardware lottery”



Expertise is scarce and tacit

- Verification and physical design dominate schedules
- Much of the craft is not written down
- Fewer architects than the demand for custom silicon



What changed: the generator got good

LLMs read specs, code, RTL and logs, and call compilers, simulators and EDA tools.

Architecture 2.0: the missing piece — environment, data, infra, self-improvement, discovery...

Hooker, “The Hardware Lottery,” CACM 2021 · Reddi et al., “Architecture 2.0,” SIGARCH 2023



The design loop is the bottleneck: too many choices, too slow to evaluate

What Does an Architect Actually Do?

What does an architect actually do?

Recall

"What do I know?"



Analyze

"What is the bottleneck?"



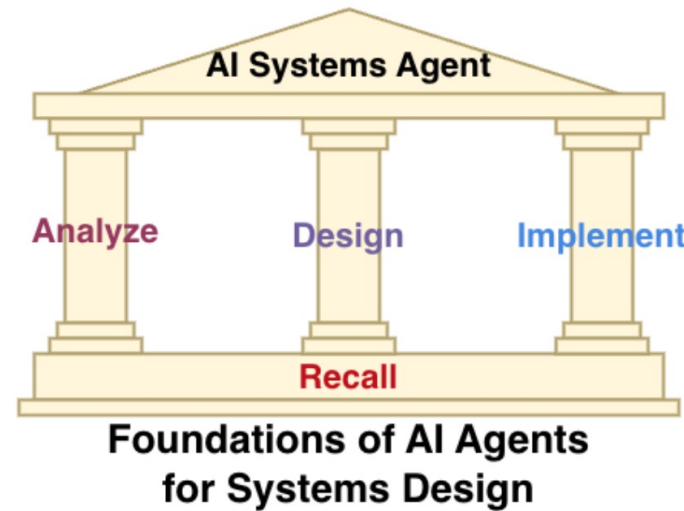
Design

"What should I change?"



Implement

"Can I realize and test it?"



Questions

Recall

What information is stored in a branch target buffer (BTB) to verify that a branch target address is a match?

Analyze

If the branch predictor and BTB make the same predictions, which will give better performance?

Design

Propose a dynamic branch predictor that integrates into the processor's pipeline.

Implement

Validate the dynamic branch predictor and BTB in a simulator and run performance benchmarks.

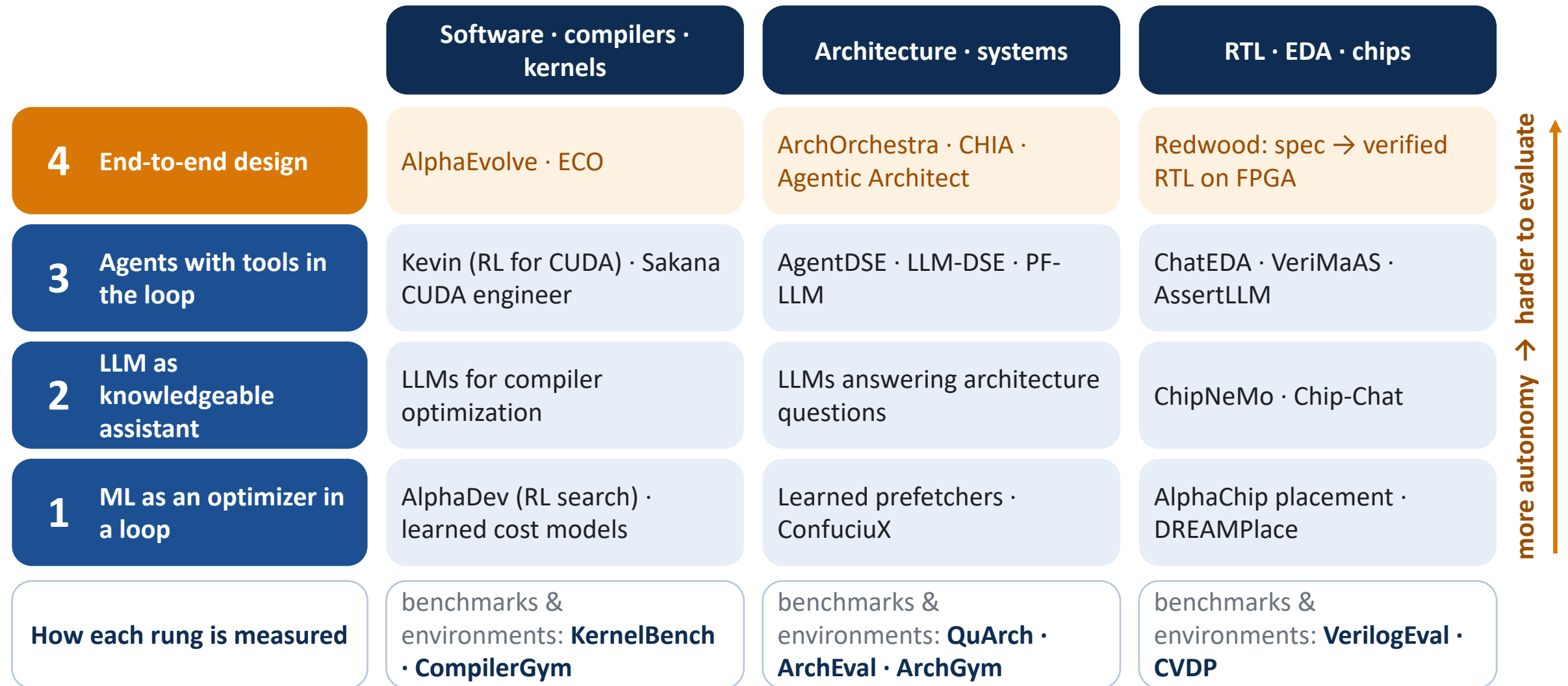
QuArch (2,671 QAs): recall → design reasoning: GPT-5 89→72 · Gemini 87→63 · Claude 86→48 · Llama 85→34

Prakash et al., QuArch, ICML 2026



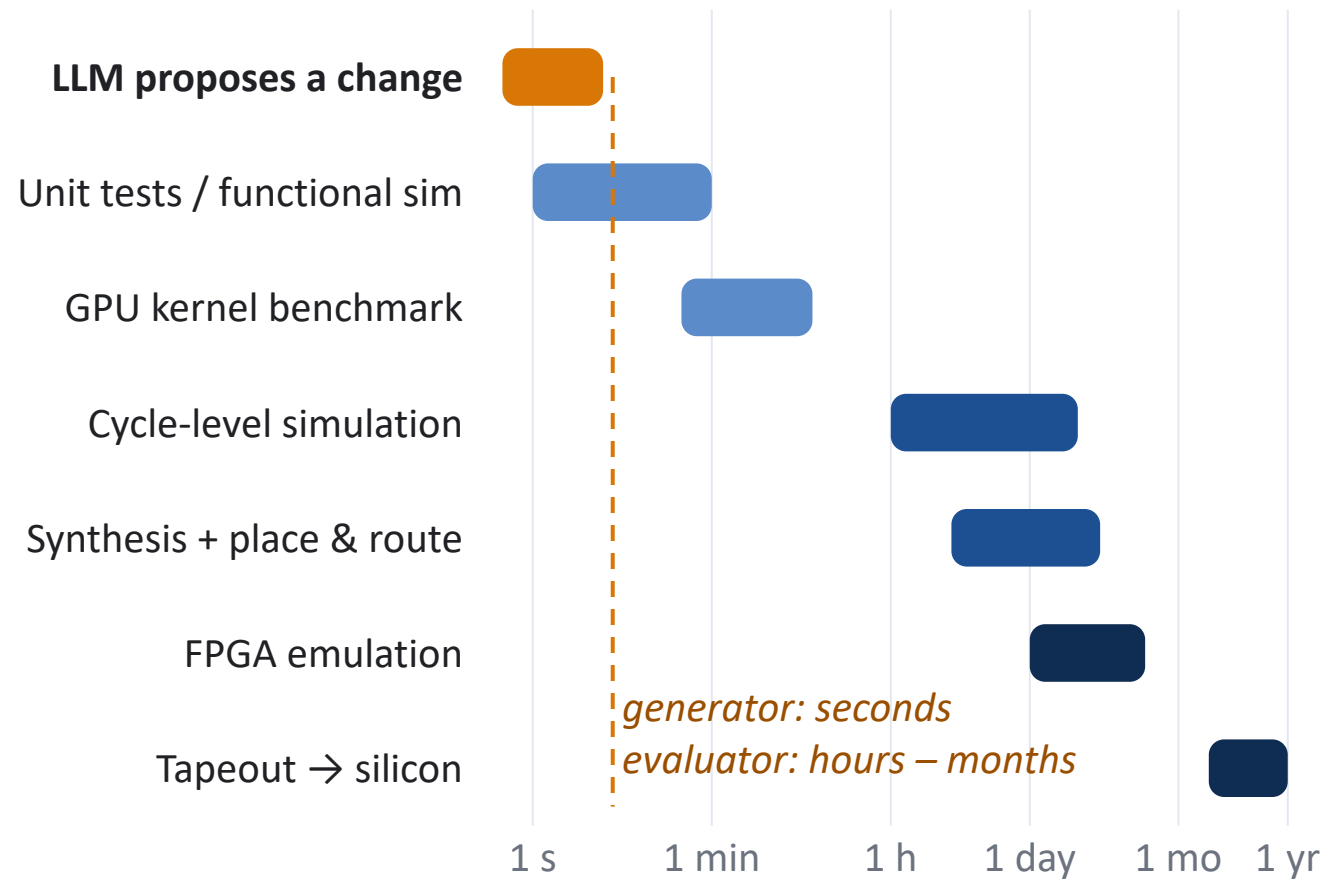
Models know the facts (85-89%) but design-reason far worse (34-72%)

A Ladder of AI for Computing



Recurring Challenge #1: The Feedback-Loop Crisis

How long until you know whether the idea was good?



C1: The feedback-loop crisis

"The generator is increasingly fast while the evaluator remains slow, expensive, noisy, or incomplete."

— GenAI for Systems, 2026



Search ≠ discovery

Rewarded only for what the evaluator can score — **the evaluator defines what AI can invent**



The generator is fast; the evaluator is slow — evaluation is the bottleneck

Four More Recurring Challenges



C2 · The tacit-knowledge problem

Design is shaped by expertise nobody wrote down: which corner cases matter, which tools to trust.

Example: valid Verilog no senior engineer would sign off on.



C3 · Trust and validation

Correctness-critical decisions — and generated artifacts look plausible whether or not they are right.

Example: the AlphaChip debate and Markov's "False Dawn" critique of its baselines.



C4 · Co-design across boundaries

Performance is decided in several layers at once; optimize one with the rest frozen leaves most of the gain behind.

Example: a kernel agent that never sees the memory hierarchy hits a local optimum.



C5 · From determinism to dynamism

Static artifacts give way to learned, adaptive policies — schedulers, prefetchers, serving plans.

Example: a scheduler great on the training trace, pathological under shifted load.



Five challenges recur from software to silicon — across 11 application areas

Module 2 Takeaways



1

A ladder, not a switch

from ML as an optimizer in a loop, to assistants, to agents with tools, to end-to-end design, each rung raises the evidentiary bar.



2

The evaluation bottleneck

the generator is fast; evaluators are slow, noisy, incomplete, and bound discovery



3

Trust comes from structure

separate proposal, evidence and realization; measured vs projected; budget-matched classical baselines; released artifacts.

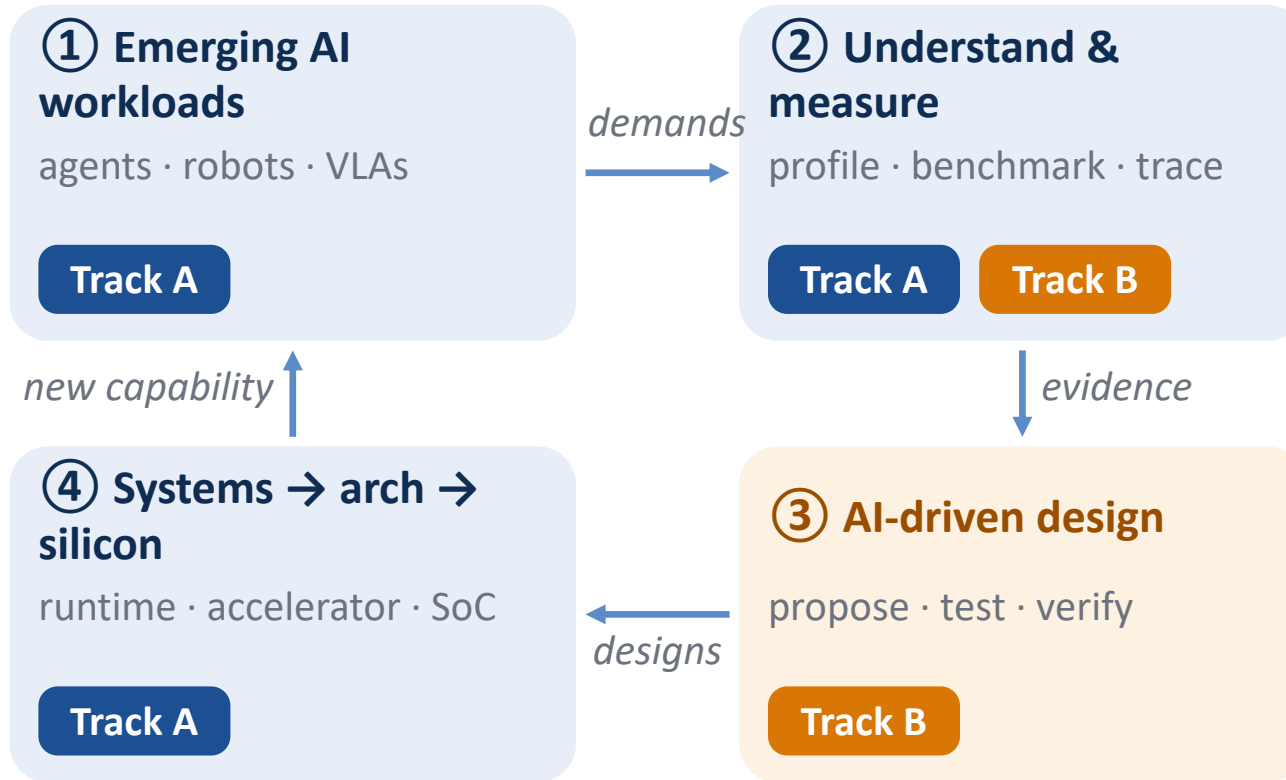
Module 2 · Weeks 8-12:

AI for compilers & kernels → AI for architecture → RTL & chip design



Two directions, one loop

The AI-Native Computing Loop



Measurement (②) is the node every project passes through

Module A · Computing for AI (① ② ④)
Profile a workload; build the runtime, mapping or accelerator; measure jointly

Module B · AI for Computing (② ③)
Turn design into an agent environment; evaluate vs classical baselines

One object, both directions
Profile where an agent's time goes (A); can an agent improve its own plan from that data (B)?

One loop — your project sits on at least two of its nodes

Key Takeaways

1

AI is a system and a loop, not a model

Workflows, control loops and reasoning graphs, with state, deadlines and feedback, are the workload we design for.

2

The bottleneck is cross-layer: co-design or stall

Decode bandwidth, KV state, orchestration gaps, real-time control, irregular kernels, fixed at the interfaces between layers.

3

AI can design computers, only as fast as we can evaluate

The generator is fast; simulators, synthesis and silicon are slow, noisy and incomplete. The evaluator bounds discovery, and measured $\neq$ projected.

4

Agent output is not evidence; measurement is

Matched baselines, budgets, ablations, held-out conditions, released artifacts, for papers and for your projects.



Next week: the measurement toolkit — roofline, tail latency, LLM inference anatomy

Next Week & Before You Leave



Week 2 · Fri Sep 18

How to Study an AI Computing System + LLM Inference Fundamentals

Required reading

- Williams et al. — Roofline (CACM 2009)
- Dean & Barroso — The Tail at Scale (CACM 2013)
- Dao et al. — FlashAttention (NeurIPS 2022)

Optional

- MLPerf Inference Benchmark (ISCA 2020)
- Sze — Efficient Processing of DNNs (2017)



Your to-do list

Now

Background survey — by Sat 11:59 PM

Mon Sep 14

Enrollment decisions (Vergil)

Fri Sep 18

Preference form · 3 readings done

Wed Sep 23

Bidding form — browse the website

This week

Finish today's 2 readings



Questions?

Background Survey



Scan to open the survey

forms.gle/7PZBZyQjFQkZ3EXt8

also on the course website



Why I am asking

- Enrollment decisions cap 30; I want a room with complementary skills
- Team formation and the project portfolio, what you can bring
- Calibrating the level of the mini-lectures



How to fill it in

- Be candid, not graded
- About 15-20 minutes; if you cannot finish now: Sat Sep 12, 11:59 PM
- Decisions Mon Sep 14 by Vergil system



Thank you — see you (I hope) next Friday, 10:10, 602 Northwest Corner